Task 4 – Production-Grade FastAPI Authentication + LLM Integration

Assigned: 3 March 2026
⏰ **Deadline:** 11:00 AM – 4 March 2026

Objective

Upgrade your existing **FastAPI project (Task 3)** by implementing a **production-ready authentication and session management system** using JWT.

This task must include:

- Secure user signup & login
- Proper password hashing
- JWT-based authentication
- Single active session per user (old session expires on new login)
- Protected endpoints
- LLM (Gemini or any other model) integration after authentication

This should reflect **real-world backend engineering standards**.

Part 1 – User Authentication System

1⃣ User Model (Database Required)

Create a proper `User` table in the database with fields like:

- id (UUID or Integer – Primary Key)
- name
- email (unique)
- hashed_password
- is_active (Boolean)
- current_session_id (String / UUID)
- created_at
- updated_at

⚠️ Email must be unique.

2️⃣ Signup Endpoint

POST `/signup`

Requirements:

- Validate input using Pydantic schemas
- Email must be unique
- Password must be hashed using:
  - `passlib` (bcrypt recommended)
- Store user in database
- Return success message (DO NOT return password)

Proper error responses:

- 400 → Email already exists
- 422 → Validation error

3️⃣ Login Endpoint (JWT Based)

POST `/login`

Requirements:

- Validate credentials
- Verify hashed password
- Generate:
  - JWT Access Token (short expiry: e.g., 15–30 minutes)
  - session_id (UUID)

JWT must include:

```
{
"sub": "user_id",
"session_id": "uuid",
"exp": "expiration time"
}
```

Store the generated `session_id` in the database inside the user record.

Return:

```
{
"access_token": "...",
"token_type": "bearer"
```

}

Error handling:

- 401 → Invalid credentials
- 403 → Inactive user

Part 4 – Single Device Login (Session Control)

This is the MOST IMPORTANT part.

Requirement:

If a user logs in from another device:

- A new `session_id` must be generated
- The previous session becomes invalid automatically

How to implement:

1. When login happens:
    - Generate new session_id
    - Update it in database
2. For every protected route:
    - Decode JWT
    - Extract `session_id`
    - Compare with database `current_session_id`
    - If mismatch → return:

401 Unauthorized – Session expired due to login from another device

This ensures:
✅ Only one active session per user
✅ Old token becomes useless

Part 5 – Protect Existing Endpoints

All existing endpoints from Task 3 must:

- Require `Authorization: Bearer <token>`
- Validate:

    - Token signature
    - Expiry
    - Session match

Invalid cases must return:

- 401 → Invalid token
- 401 → Expired token
- 401 → Session expired
- 403 → Inactive user

Part 6 – LLM Integration (Gemini / OpenAI / Any Model)

After login, create a protected endpoint:

POST `/ask-llm`

This endpoint must:

- Require authentication
- Accept user prompt
- Call external LLM API (Gemini / OpenAI / any model)
- Return AI-generated response

Example flow:

1. User logs in
2. Receives JWT
3. Sends request:

   Authorization: Bearer <token>

4. Backend:
    - Validates JWT
    - Confirms session
    - Calls LLM
    - Returns AI response

LLM Integration Requirements

- API key must be stored in `.env`
- Never hardcode secret keys
- Use proper exception handling
- Handle:
  - API errors
  - Timeout
  - Invalid responses

🧪 Testing Requirements

Students must test:

- Signup with duplicate email
- Login with wrong password
- Expired JWT
- Login from second device → first token invalid
- Access protected route without token
- Access protected route with expired token
- LLM endpoint works only after login

Expected Project Structure

```
app/
├── main.py
├── database.py
├── models.py
├── schemas.py
├── auth.py
├── routes/
│   ├── auth_routes.py
│   ├── protected_routes.py
│   ├── llm_routes.py
├── core/
│   ├── config.py
│   ├── security.py
```

⭐ Bonus (Optional – Extra Marks)

- Implement refresh token mechanism

- Add logout endpoint that invalidates session
- Add role-based access (admin/user)
- Dockerize project
- Add rate limiting

Evaluation Criteria

Criteria	Marks
Proper password hashing	10
JWT implementation	15
Single session enforcement	20
Protected routes	15
LLM integration	15
Error handling	10
Code structure	10
Clean architecture	5

Final Deliverable

- Working FastAPI project
- `.env` configuration
- Database migrations
- Postman collection or Swagger testing
- README with:
  - Setup steps
  - How session invalidation works
  - How LLM is integrated